Final Report

Names of Group Members

CMSC 495 – Section XXXX

DATE

Table of Contents

1. Overview *3*

2. Project Plan *4*

3. Requirements Specification *4*

4. System Specification *4*

5. User's Guide *4*

6. Test Plan and Results *5*

7. Design and Alternate designs *5*

8. Development History *8*

9. Conclusions *11*

9.1. Lessons Learned **11**

9.2 Design Strengths **12**

9.3 Limitations **13**

9.4. Suggestions for Future Improvement **13**

1. Overview

The Plate License and Tag Editor (PLaTE) application was designed to automate the license plate management division of the DMV. Features provided by the application include:

- License plate data management including adding and deleting records
- Renewals of existing license plates
- Ordering new license plates
- Sending payment information to credit card companies securely
- Storage of application data including customer data, vehicle data, license plate data

The application operates though a Python based Graphic User Interface (GUI) with a SQL based database for information storage. The application runs on either Windows or Mac systems that have python and SQLite loaded on the system.

The application has 3 major components, the GUI, the database, and the backend code. Development of the system was accomplished by a two-person team (Pearce, Rosario-Baez). Being that the development team had a limited manpower pool, the entire development team were involved almost every of the application management and oversight of the major portions of the development were assigned to specific individuals as follows:

- GUI development - Name 1
- Database management - Name 2
- Python application backend code – Name 3
- Documentation - Name

2. Project Plan

The finalized Version 3 of the Project Plan is embedded in this document. Open it by clicking on the icon below.

3. Requirements Specification

Requirement specifications are covered in the finalized project plan. Access it by clicking on the icon in the previous section.

4. System Specification

The application will come with the Python GUI, SQL code to place the required tables on any SQL database, and the Python code to run the GUI/SQL database integration. The Python GUI will communicate with the SQL database to add, view, and delete customer records. These records will be used to keep track who owns a vehicle in the state of Maryland and help Maryland drivers renew and obtain new license plates to maintain proper ownership of the vehicle.

5. User's Guide

The PLaTE Version 3 User Guide is a comprehensive document that has been vetted by the team and the professor on through multiple revisions. The current revision is a clearly laid out step-by-step guide to every function of the PLaTE program. Each process contains figures of each step of

the process to guide an inexperienced user through the program. The finalized Version 3 of the User Guide is embedded in this document. Open it by clicking on the icon below.

6. Test Plan and Results

The test plan documents, and results have been key to ensuring that the release of our program was successful. Combing the GUI and associated code step by step with the test pans have helped us to identify problems that would have made it into production. All problems found have been fixed until the test plan could be ran with no failures

The Finalized Test Plan and its attachments are embedded in this document. Open them by clicking on their icons below.

7. Design and Alternate designs

Primary designs for the program were comprehensive, the original program UML is shown below:

UML

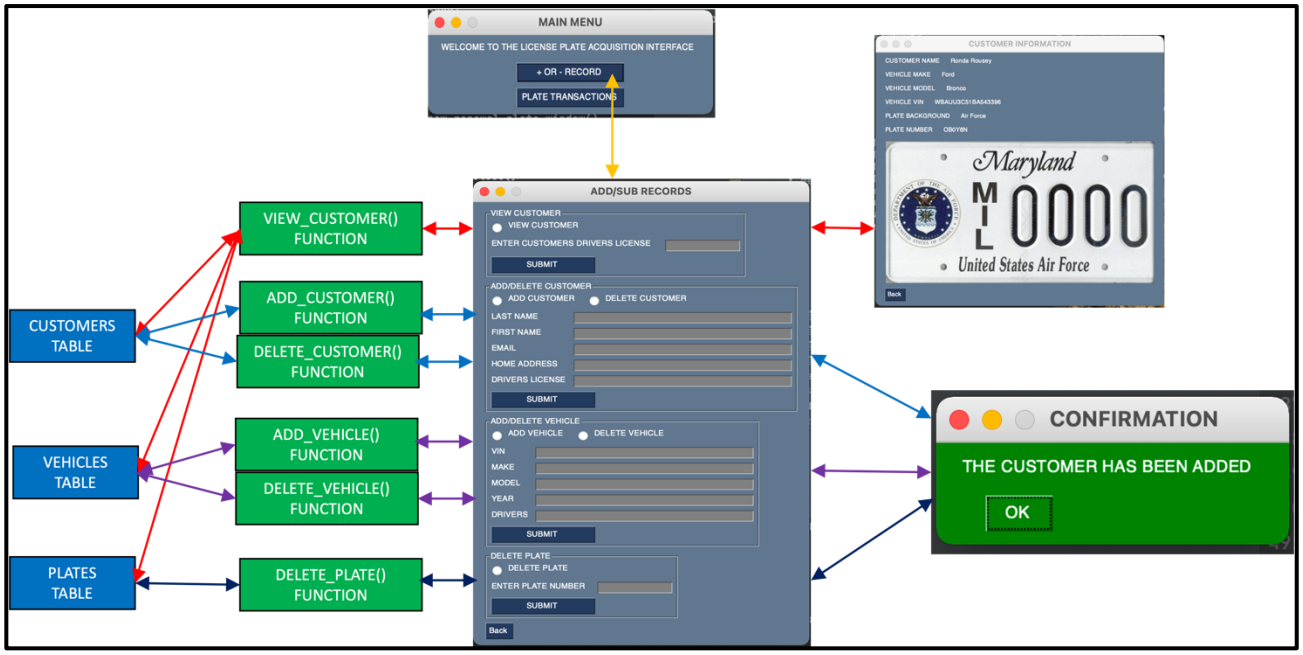

Figure 1. UML Part 1

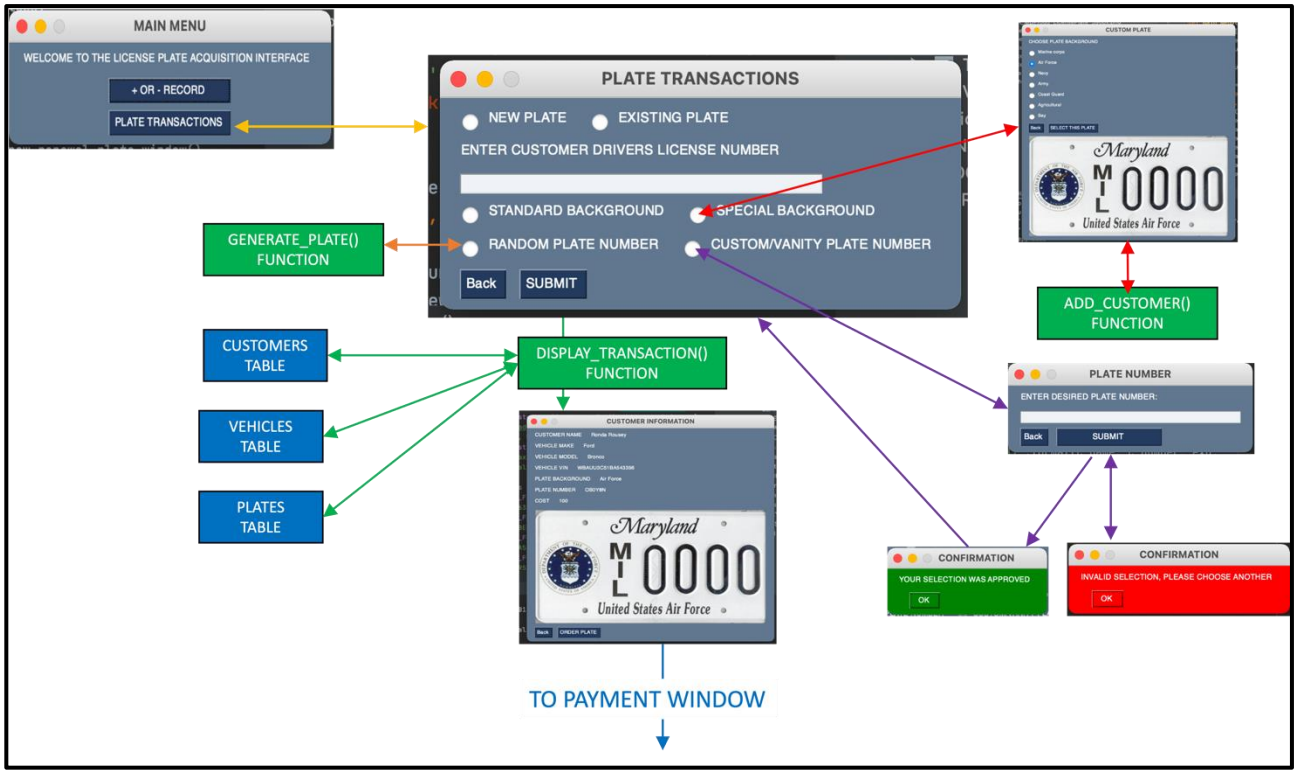

Figure 2. UML Part 2

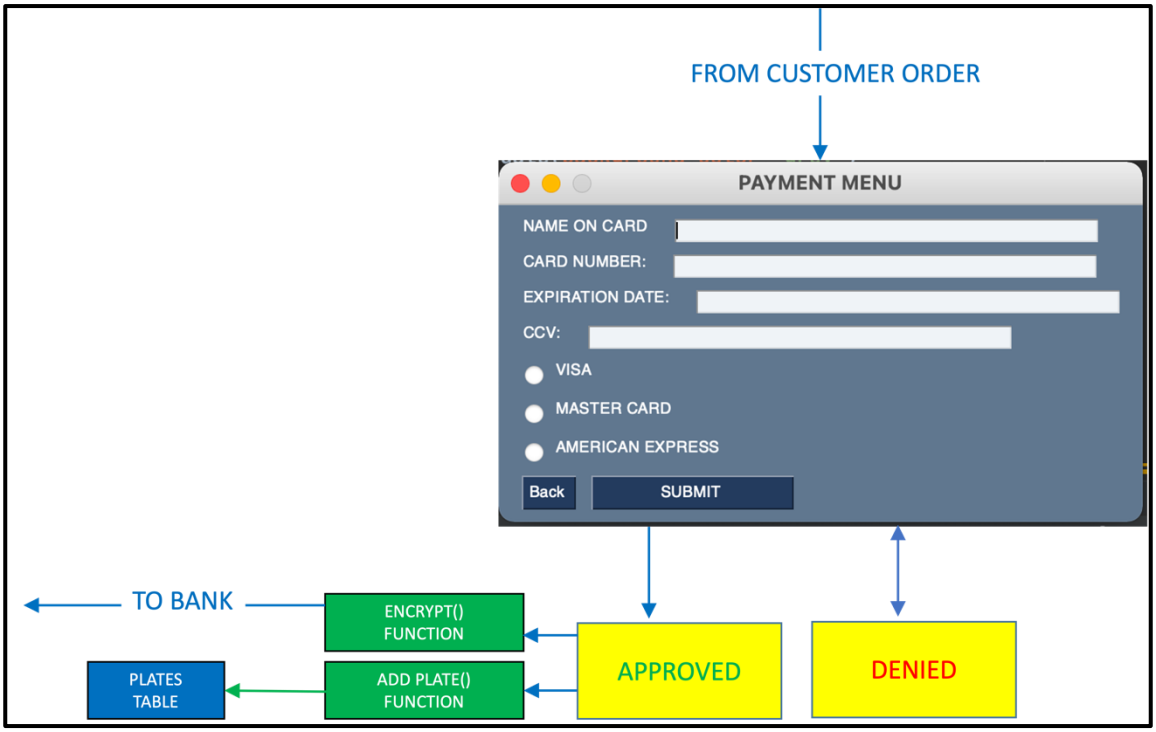

Figure 3. UML Part 3

Although this fix helped the user to be able to easily see records, we quickly realized that we did not take into consideration that a customer could own more than one car and could there for have multiple cars and plates that would have to be connected and able to be accessed individually for display and renewal. purposes. The solution to that was several functions and a new window. The new functionality of the program made the program look to see if a customer had more than one vehicle when their driver's license was entered in the normal process. If there was only one record everything processed as normal. If there was more than one the window below would open showing the VIN, make, model, and year for all the vehicles associated with the driver's license entered.

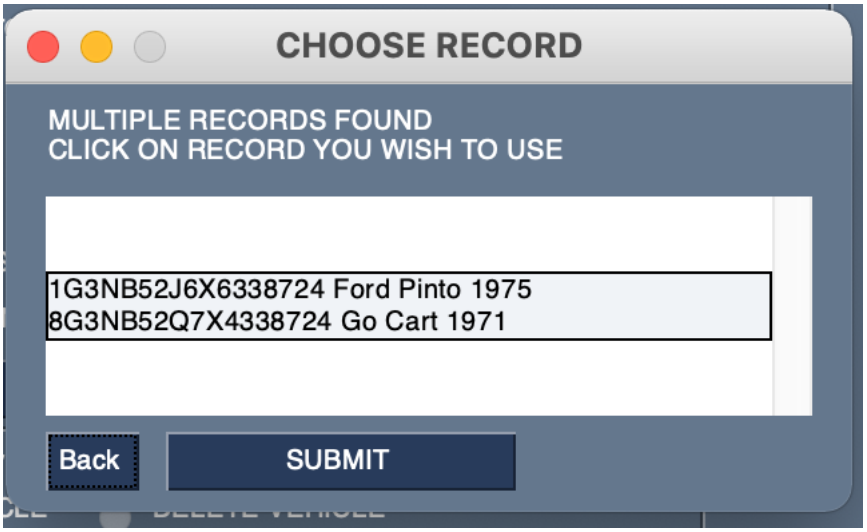

Figure 4. Multiple Record User Interface

The user could then choose which record they wished to use by clicking on the record in the displayed list and pressing submit. The record that was selected would then process. This window and the functionality attached it fixed a major shortfall in the program. The program remains in its final state with only the addition of this one window and popups to tell users the success or failures of their entries.

The Finalized Design Plan is embedded in this document. Open it by clicking on their icons below.

8. Development History

The Plate program s developmental history is best displayed through our milestone chart and later quite comprehensively through our phase submissions. On the milestone chart shown below, the **black lines** show the milestones as they were set at the beginning of the project. The

green lines show the actual times. Most time frames were very close to the prospective times set forth at the beginning. Times for GUI assembly, backend coding, and connection of components are shown longer as the GUI and its functionality was an ongoing process after the initial build for the entire project. All of these times ran over their estimates but concluded in plenty of time while leaving a much better program than in the beginning.

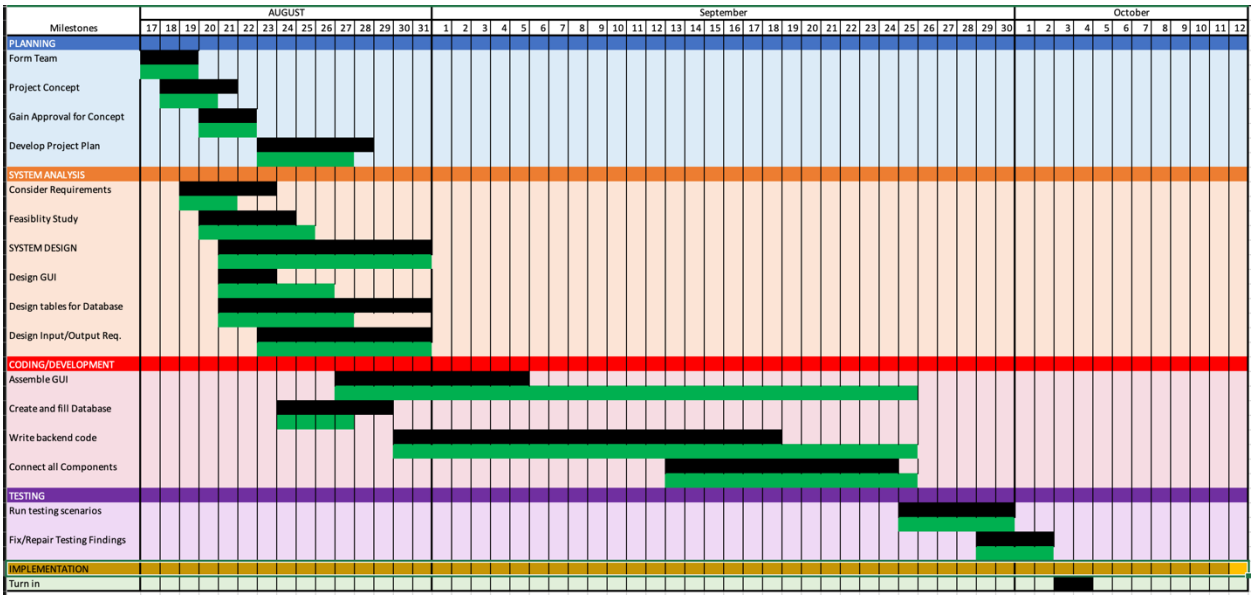

Figure 5. Milestones of the Project VS Actual Times

An overview of the three phases of this developmental processes where as follows:

PHASE I (5 Sept 15 – Sept 21)

Name Got to work on learning PYSimpleGUI and getting the initial build of the GUI together. This was a major undertaking as neither member of the team had ever used Python to write a GUI.

Name started writing pseudo functions for the backend and creating SQL tables that would hold the data needed for the program and hold enough information on each customer, vehicle, and license plate so that they could be attached to their corresponding records and be used by the GUI and backend code.

PHASE II (Sept 22 – Sept 28)

Namecontinue to work on the GUI, adding functionality while coordinating with Pearce to ensure that the backend code was written to correspond with the database as it was coming together.

Name added 20 rows of values to each table to use when constructing the backend code. Pearce then wrote code into each function, testing that each function worked with the SQL database adding, deleting, and viewing the information from the database.

PHASE III (Sept 29 – Oct 5)

The program is functionally complete at this point and bugs are being discovered as we wade through testing. At this point in the project both member of the team had to be very careful of version control. Coding was often done on Zoom with both member working through any problems that had arisen.

9. Conclusions

Over the past eight weeks, team one has put their shoulder to the wheel and fought through some difficult learning curves. Despite all that, we feel that the product we produced is hearty and does what we want it to. We did not, at any time during the process, say to ourselves, “let’s skip this functionality so that we can get this thing done”. We fought through some areas, but all aspects of the program’s functionality were as we envisioned them and some even turned out better than we had expected. Both team members are much better programmers and project coordinators because of the programming tasks and teamwork involved in this project.

9.1. Lessons Learned

There are several lessons learned during this project:

First, be flexible. There will almost always be design changes that must happen once you start coding. They may be small, but they may affect other parts of the program. Accept that this is going to happen and don’t try to stall or hide them. Triage them as they are found and decide what needs to be done to make things right. We ran into a few such items in our code as we went along. Either a program logic item that we had not considered or one that we had considered but when applied acted differently. We discussed fixes when the problems were identified and got after it. We are proud to say that there is not one problem, that we identified that, that we did not fix.

Next, keep to your timelines if possible. We set milestones and either kept or exceeded them. This allowed us time to meet incoming problems head on. Not only in our code but in our home life and jobs. Because we stayed a focused and did not let ourselves get behind on the

coding or the documentation, we were able to overcome not only unseen code problems but also events that were out of the team's control that took a team member offline for a day or two.

Lastly, the answers are out there. We are fortunate today to have a vast repository of knowledge at our fingertips. If you wish to learn or clarify something. The answer is most often just a internet search away. We were lacking some of the knowledge we needed to complete this project at the beginning. We identified that fact early and taught ourselves what we needed with many hours of research. The lesson learned is that you can do almost anything with coding today, it may take some research, but it is possible.

9.2 Design Strengths

Our program is solidly built and held together with strong code. Lines of code that were used more than once were made into functions so that they could be managed easier, and the code would be easier to read from an outside perspective. We did not allow for covering or burying flaws that we found while testing. If a rewrite of a section of the code was required to fix issues raised by testing, it was rewritten. This would often set us back, but we had made provisions in the planning for this and stayed with or ahead of our timeline as a rule.

We designed the program to be easy to use. Sections of the GUI that require use input are highlighted when that input is needed. All other input areas are shaded out. Popup boxes notify users when they are using the program incorrectly or when errors are made during data input. Visual examples are given of plates as they are ordered aid the user and customer to verify what they are ordering.

The GUI is easily modifiable. Future upgrades to the program could easily modify, replace, or add windows in response to user feedback. This will allow the software development cycle to continue easily as it comes around in its cyclical manner.

9.3 Limitations

As far as limitations go for the program, our team focused on removing limitations when they were discovered in the design process or when coding started. That is not to say that our program does not have limitations. It means that we have far fewer limitation than we would have if we would not have worked under that credo. There are limitations both remaining both programmatically and structurally.

- The SQLite database will only hold approximately 281 terabytes of data. (SQLite.org, 2021)
- Each table in the database can only hold 18,446,744,073,709,551,616 rows of data (SQLite.org, 2021)
- We currently do not automatically track expiration dates in the database. This must be done manually and could be made automated in future releases of the program
- Current versions of Python and SQLite must be loaded on each machine that wishes to use the program.

9.4. Suggestions for Future Improvement

Our team has kept an eye out for future improvements as the program was coming together. Some of those improvements that were identified we were able to incorporate immediately. Others were noted for the future. One of the upgrades we were able to incorporate was the ability

to select which record was being selected when the customer hand more than one vehicle. How the user performed this task was upgraded from a manual count of the records by the user to a list populated with the options available that the user can click on their selection and submit it. There was a lot of blood sweat and tears to bring that upgrade into production, but we accomplished it and it works beautifully.

Some of the future improvements that we identified for future versions of the program was the addition of more plate backgrounds beyond the eight versions we currently offer, an expiration date tracker that automatically combs the database and provides that information to the user. This task currently must be done manually. Lastly, combing the program into an executable file would make the initial download and startup of the program easier and could be accomplished by an average user instead of a programmer.

REFERENCES

Geeks for Geeks.org. (2019, March 27). *Python | Convert list of tuples to list of list*. GeeksforGeeks. https://www.geeksforgeeks.org/python-convert-list-of-tuples-to-list-of-list/

Programiz. (n.d.). *Python list remove()*. Programiz: Learn to Code for Free. https://www.programiz.com/python-programming/methods/list/remove

PySimpleGUI. (2021). *PySimpleGUI*. https://pysimplegui.readthedocs.io/en/latest/

Python, R. (2020, June 17). *PySimpleGUI: The simple way to create a GUI with Python – Real Python*. Python Tutorials – Real Python. https://realpython.com/pysimplegui-python/

Python Software Foundation. (2021). *Sqlite3 — DB-API 2.0 interface for SQLite databases — Python 3.9.7 documentation*. 3.9.7 Documentation. https://docs.python.org/3/library/sqlite3.html

SQLite.org. (2021, June 18). *Implementation limits for SQLite*. SQLite. https://www.sqlite.org/limits.html

Trinket.io. (2021). *The basics of the pysimplegui program*. https://pysimplegui.trinket.io/demo-programs#/demo-programs/the-basic-pysimplegui-program

Tutorialspoint. (2021). *SQLite - Python*. Biggest Online Tutorials Library. https://www.tutorialspoint.com/sqlite/sqlite_python.htm

Tutorialspoint. (2021). *SDLC - Overview*. The Biggest Online Tutorials Library - AppML, AI with Python, Behave, Java16, Spacy. https://www.tutorialspoint.com/sdlc/sdlc_overview.htm

W3 Schools. (2021). *SQL SELECT statement*. W3Schools Online Web Tutorials. https://www.w3schools.com/sql/sql_select.asp